

StreamGuard: Low-Overhead Resilience for Real-time HPC Data Streams

Hai Duc Nguyen
Argonne National Laboratory
Lemont, IL, USA
University of Chicago
Chicago, IL, USA
hai.nguyen@anl.gov

Bogdan Nicolae
Argonne National Laboratory
Lemont, IL, USA
bnicolae@anl.gov

Tekin Bicer
Argonne National Laboratory
Lemont, IL, USA
tbicer@anl.gov

Amal Gueroudji
Argonne National Laboratory
Lemont, IL, USA
agueroudji@anl.gov

Matthieu Dorier
Argonne National Laboratory
Lemont, IL, USA
mdorier@anl.gov

Kyle Chard
University of Chicago
Chicago, IL, USA
chard@uchicago.edu

Ian Foster
University of Chicago
Chicago, IL, USA
Argonne National Lab
Lemont, IL, USA
foster@uchicago.edu

Abstract

Real-time scientific workflows operate on continuous data streams and must produce timely, high-quality results despite executing on complex, failure-prone infrastructure. Hardware faults, network disruptions, and performance anomalies caused by resource contention or system heterogeneity can severely degrade performance and violate real-time constraints. We focus on strengthening the resilience of the producer-consumer streaming pattern, a fundamental building block of scientific streaming workflows. We present two complementary techniques: (i) a dynamic, asynchronous, non-blocking checkpointing mechanism that preserves progress without interrupting computation, and (ii) a progress-aware load redistribution strategy that detects slow workers and proactively rebalances tasks. Together, these mechanisms maintain forward progress and balanced execution even in highly error-prone environments. Experimental results show that our approach reduces the impact of failures and performance anomalies by up to 6×, while introducing less than 1% overhead in failure-free execution.

CCS Concepts

• **Hardware** → **Fault tolerance**; • **Information systems** → *Data streaming*; • **Software and its engineering** → **Checkpoint / restart**.

Keywords

Resilience, Stream Processing, Message Logging, Load Balancing

This paper is authored by an employee(s) of the United States Government and is in the public domain. Non-exclusive copying or redistribution is allowed, provided that the article citation is given and the authors and agency are clearly identified as its source. Request permissions from owner/author(s).

ICS '26, Belfast, United Kingdom

2026. ACM ISBN 979-8-4007-2522-7/26/07

<https://doi.org/10.1145/3797905.3807872>

ACM Reference Format:

Hai Duc Nguyen, Bogdan Nicolae, Tekin Bicer, Amal Gueroudji, Matthieu Dorier, Kyle Chard, and Ian Foster. 2026. StreamGuard: Low-Overhead Resilience for Real-time HPC Data Streams. In *2026 International Conference on Supercomputing (ICS '26)*, July 06–09, 2026, Belfast, United Kingdom. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3797905.3807872>

1 Introduction

Modern scientific instruments generate large volumes of data as a continuous stream that must be processed in near real-time to guide ongoing experiments. Examples include synchrotron radiation facilities [19, 35, 47, 53], large-scale sensor deployments [31], and online scientific simulations [28, 55], where data are continuously acquired and processed using multi-stage workflows to produce timely feedback. Unlike traditional batch-oriented scientific workflows, these applications operate on unbounded data streams and must continuously produce meaningful results within strict time constraints [4, 30]. As a result, scientific streaming workflows have emerged as a critical paradigm for enabling real-time scientific discovery [41, 44].

However, achieving reliable execution for such workflows remains challenging. These workflows are typically deployed on large-scale, distributed infrastructures where failures and performance variability are unavoidable. Fail-stop events, such as process crashes or node failures, interrupt execution and can force costly recomputation if progress is not preserved. Additionally, even if processes do not abruptly end due to fail-stop events, they may be subject to performance degradation due to various other anomalies: communication delays and timeouts, transient interference due to competition for shared resources, misconfigured software stacks, etc. Even if such anomalies are localized, their effect can quickly propagate in the workflow due to dependencies, leading to quality-of-service loss and/or increase in end-to-end latency. These

challenges become increasingly severe as the system scale grows, where failures and performance variability become the norm rather than exception [13, 22].

Limitations of state-of-the-art. Traditional HPC resilience techniques emphasize global checkpointing and recovery [20], which is appropriate for tightly coupled, bulk-synchronous simulations. In this case, there is an implicit assumption that a safe point can be eventually reached (e.g., end of iteration), where a global barrier ensures there are no more in-flight messages, and simplifies the capture of a globally consistent state as a set of individual checkpoints corresponding to each process. However, stream processing workflows are typically modeled using producer-consumer patterns that are loosely coupled. Since the data are constantly in motion, identifying an opportunity to capture individual checkpoints may not be possible or may introduce excessive coordination overheads. Conversely, many stream-processing frameworks, such as Flink [14], are not designed to run on HPC machines. They provide built-in resilience techniques tightly coupled with their execution models [14, 54] and are often too conservative (e.g., based on Chandy-Lamport [15] algorithm), thereby incurring high overheads. This limits flexibility and makes them difficult to adopt in specialized scientific workflows. Furthermore, such techniques typically optimize for throughput [45] or per-partition latency [25] rather than prioritizing end-to-end latency. Thus, there is a need to develop dedicated resilience solutions that overcome these limitations.

Key insights and contributions. In this paper, we address this gap by rethinking resilience from the perspective of scientific streaming workflows. Instead of enforcing workflow-wide coordination or embedding resilience into a specific framework, we design resilience mechanisms as modular components operating at the producer-consumer level. This design leverages the embarrassingly parallel nature of scientific streaming workflows, allowing failures and performance anomalies to be handled independently with minimal coordination. By decoupling resilience from execution frameworks and avoiding global synchronization, our approach enables resilience to be introduced without sacrificing scalability or real-time performance.

We present StreamGuard, a resilience solution for real-time scientific streaming with two complementary techniques: (i) *dynamic, asynchronous, non-blocking* checkpointing for low-overhead failure recovery, and (ii) *progress-aware load redistribution* that detects slowdown anomalies and rebalances data to satisfy real-time constraints. While each builds on established ideas, our contribution is their integration into a *unified, modular* design at the *producer-consumer boundary*, jointly mitigating failures and slowdowns without the workflow-wide coordination required by prior approaches.

We evaluate StreamGuard using a real-world scientific streaming workflow for tomographic reconstruction used at synchrotron radiation facilities [10, 12]. Experimental results demonstrate that StreamGuard reduces the impact of failures and slowdown anomalies by up to 6× while introducing less than 1% overhead in failure-free execution. Across a wide range of scales, failure rates, and anomaly conditions, StreamGuard’s design consistently maintains processing time close to ideal execution and enables workflows to meet real-time deadlines where existing approaches fail. We summarize our contributions as follows:

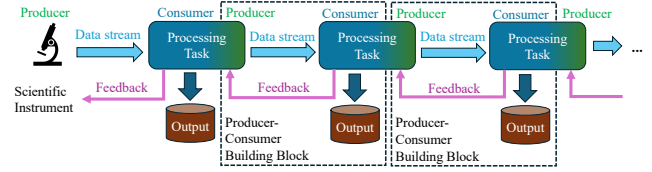


Figure 1: Scientific streaming workflow is formed by a sequence of producer-consumer building blocks

- (1) A unified, decoupled producer-consumer-centric resilience design that integrates dynamic non-blocking checkpoint-and-retry with progress-aware load balancing as composable modules, avoiding workflow-wide coordination, reducing performance degradation from failures and slowdowns by up to 4× and 5×, respectively (§5).
- (2) A modular resilience architecture for real-time scientific streaming that allows applications to selectively enable resilience mechanisms as needed. When combined, these modules effectively prevent compounded performance degradation, with performance increase to at most 40% under extreme failure and anomaly conditions at scale (§6).
- (3) A systematic evaluation using a real-world scientific streaming application that demonstrates improved robustness, scalability, and performance transparency compared to state-of-the-art HPC and stream-processing resilience approaches, while consistently meeting real-time constraints in scenarios where existing solutions may fail (§7).

2 Background

We consider a large class of streaming scientific applications that follow a producer-consumer pattern as illustrated in Figure 1. The workflow begins at a scientific instrument, such as a synchrotron radiation facility, that continuously generates large volumes of data during experiments. These data are transferred to HPC systems for processing through a sequence of stages, where each stage consists of multiple parallel workers processing different portions of the data stream (i.e., partitions).

2.1 Producer-Consumer Streaming

The communication between the stages follows a producer-consumer model. Upstream workers act as *producers* by emitting data records, while downstream workers act as *consumers* that receive and further process incoming data records. Depending on application requirements, the data records may be distributed among consumers through different patterns, such as shuffle, where records are scattered across consumers, or broadcast, where every record is delivered to all consumers. Regardless of the distribution pattern, data flowing between producers and consumers is typically divided into sequential *partitions* with unique identifiers (or keys). These partitions can be processed independently, enabling efficient parallel execution over large-scale data streams.

After processing, consumers may generate intermediate outputs, persist state, or provide feedback to upstream stages. The outputs are then forwarded downstream, where the same producer-consumer interaction repeats. Because most computation and data movement occur at these boundaries, the producer-consumer

pair forms the primary unit where performance variability, failures, and anomalies directly affect end-to-end workflow behavior.

2.2 Key Characteristics

Building on the producer-consumer pattern described earlier, scientific streaming workflows exhibit characteristics that distinguish them from both traditional HPC and stream-processing workloads.

Unbounded data stream. Data are continuously generated by instruments or edge devices as an unbounded sequence of records. Each record captures only a small portion of the evolving experiment, and the full outcome emerges gradually as more data arrive. This differs fundamentally from traditional HPC workloads, where the complete dataset is typically available in a parallel file system before processing begins. In scientific streaming, workers must operate on incomplete and continuously evolving data.

Iterative refinement. Because data arrive incrementally, early outputs are necessarily approximate. As additional data are consumed, workers iteratively refine their results, steadily improving accuracy and quality. This pattern is common in applications such as iterative reconstruction [10, 40] and long-term campaigns [6, 33, 51]. Unlike conventional stream-processing systems that handle many independent, small, and fine-grained records, scientific streams often consist of coarse-grained, high-volume data units that collectively represent meaningful scientific states.

Embarrassingly parallel processing. Streamed partitions can be processed independently by many parallel workers with minimal coordination. As a result, scientific streaming workflows often rely on workers deployed across multiple compute nodes, leveraging their collective power to accelerate stream processing.

Real-time quality constraints. Although the stream is continuous, processing must produce useful results within practical time limits imposed by system policies or experimental needs. Therefore, the goal is not merely to process all incoming data, but to maximize output quality within a bounded execution window—a notion often captured as Quality of Service (QoS).

3 Problem Formulation

Table 1: Notations used in this paper

Notation	Definition
s_i	A partition of the data stream
N	Number of data partitions
k_i	Data partition identifier (or key)
w_j	Workers to process data partitions
M	Number of workers
$p_i(t)$	Progress of a data partition s_i at a time t
$q_j(t)$	Progress of a worker w_j at a time t
L	Real-time deadline
P	Target progress before reaching the deadline
μ	Per-worker mean time to failure

This paper addresses the **resilience challenges** of the producer-consumer pattern. We model data stream S between a producer and

a consumer as a set of independent partitions $S = \{s_1, s_2, \dots, s_N\}$ where a partition s_i is the set of records that need to be processed together in a sequential order (e.g., data collected by a single device over time) identified by a unique key k_i . Let $w_1, \dots, w_M$ denote the available *workers* (e.g., processes or compute nodes) allocated to the consumer to process data. We assume $M \leq N$, meaning that each worker may process multiple partitions. We also assume that data partitions have similar computational demand, which can be achieved through appropriate data partitioning methods [34, 52]. In contrast, workers are not assumed to be identical, as their performance may vary due to hardware differences, resource contention, or dynamic runtime conditions. To flexibly work around this heterogeneity, we allow data partitions to be reassigned from one worker to another and let $a(s_i, w_j, t)$ be an indicator function specifying whether the data partition s_i is assigned to the worker w_j at time t :

$$a(s_i, w_j, t) = \begin{cases} 1, & \text{if } s_i \text{ is assigned to } w_j \text{ at } t \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

We assume time is discretized into time units in which partition-to-worker assignment remain unchanged. Since consumer workers process data in an iterative manner, we define *partition progress* $p_i(t)$ as the number of records that have been processed over the data partition s_i by the time t . Likewise, *worker progress*, $q_j(t)$, is the total progress a worker w_j has made on its assigned partitions:

$$q_j(t) = \sum_{i=1}^N \sum_{\tau=1}^t (p_i(\tau) - p_i(\tau-1)) a(s_i, w_j, \tau) \quad (2)$$

Compared to traditional batch processing, streaming workflows can produce usable outputs without waiting for the complete dataset. Initial results can be generated after a few processing iterations and progressively refined as additional data arrive. This property makes streaming well-suited for real-time applications, where consumers must produce meaningful outputs within a specified time window. We model this real-time requirement using a deadline L and a progress target P . The real-time constraint is satisfied if all data partitions reach at least progress level P by the deadline L :

$$\forall i : p_i(L) \geq P \quad (3)$$

The notations used in this formulation are summarized in Table 1 and are used throughout the remainder of the paper.

The producer-consumer pattern often requires massive parallelism to process large data streams, so the workflow can be highly sensitive to failures and performance anomalies. We consider two common classes of disruptions.

Fail-stop events. A fail-stop event occurs when a worker abruptly stops execution due to errors such as segmentation faults or out-of-memory conditions [49]. Fail-stop failures are often modeled as random processes [8], where workers fail independently with identical probabilities. These failures are characterized by the mean time to failure (MTTF), denoted as μ , which represents the average time a single worker runs before failing. Worker failure probability is typically described using an exponential distribution $\text{Exp}(\lambda)$, where $\lambda = \frac{1}{\mu}$ represents the failure rate [20]. The mean time between failures (MTBF) for the whole consumer is given by:

$$\mu_P = \frac{\mu}{M}. \quad (4)$$

Equation 4 shows that increasing parallelism reduces MTBF. This is intuitive, as more workers introduce more potential points of failure, causing failures to occur more frequently. This phenomenon poses a significant challenge for achieving resilient execution at large scales. For instance, if the MTTF of a single worker is one day, a consumer with 1000 workers would, on average, fail in less than 2 min—often not long enough for the consumer to provide any meaningful results [10]. Moreover, even with workers running on infrastructures designed to operate reliably for years, the sheer scale of large systems amplifies failure rates. Studies have shown that large-scale deployments, with thousands of compute nodes, can experience failures several times per day due to the aggregated likelihood of individual node failures [13, 23].

Anomalous performance degradation. Not all performance incidents result in complete worker failures. In many cases, long-lasting conditions such as network congestion, resource contention, hardware interference, or inherent system heterogeneity reduce worker performance but do not stop execution. As a result, data streams assigned to affected workers progress more slowly than others. We refer to this phenomenon as a *slowdown anomaly*. Because real-time execution requires *all* streams to reach a minimum progress threshold (as defined in Equation 3), overall consumer performance becomes dominated by the slowest stream.

Slowdown anomalies can have severe consequences if left unmitigated. Empirical studies of large-scale distributed and cloud systems have shown that worker execution times commonly exhibit heavy-tailed or power-law-like distributions under heterogeneous environments [17, 48]. In such settings, the majority of workers progress at similar speeds, while a small fraction experiences disproportionately longer execution times. This behavior is widely known as the *straggler phenomenon* [1, 18], where a few slow workers dominate overall completion time despite representing only a small portion of the system.

The impact of slowdown anomalies is particularly pronounced in scientific streaming workflows. Since outputs are only meaningful when progress across all partitions reaches a required level, the slowest stream effectively determines whether the real-time deadline can be satisfied. As system scale increases, the probability of encountering slow workers also increases, amplifying the likelihood that slowdown anomalies degrade overall performance. These characteristics make anomaly mitigation a critical requirement for maintaining reliable real-time execution in scientific streaming environments.

4 Related Work

Related work has broadly addressed resilience for fail-stop failures or other performance degradation anomalies, both in the context of tightly coupled and loosely coupled workflows.

Coordinated Checkpoint-Restart. HPC resilience is dominated by the checkpoint-restart (C/R) paradigm [9], necessitated by the tightly coupled nature of parallel applications. Because such applications typically use MPI for inter-process communication, the failure of a single process invalidates the global state and requires all nodes to roll back to a coordinated recovery point. Recent research

has shifted from global blocking checkpoints toward multilevel hierarchies that adapt to failure severity, spanning local memory tiers (GPU HBM, host memory, SSDs), erasure coding, buddy replication, burst buffers, and external object stores such as DAOS [37]. Prominent examples in this space are VELOC [42, 43], SCR [39], FTI [7], and ADIOS [46]. A key optimization is to enable asynchronous capture and flushing of checkpoints in order to hide I/O overheads and various other transformations (data aggregation [26], compression [38], etc.). However, despite extensive work in reducing checkpoint overhead, these approaches implicitly assume a safe point at which a global barrier ensures no in-flight messages exist (e.g., end of iteration). In stream-processing workflows, such a safe point does not naturally exist and forcing one requires expensive synchronization (§2.2), making global C/R infeasible.

Resilient Stream Processing. Conversely, stream-computing resilience leverages the loosely coupled nature of producer-consumer patterns through checkpoint-replay mechanisms derived from the Chandy-Lamport algorithm [15]. Unlike HPC’s rigid synchronization, stream processors such as Google Cloud Dataflow [25] and Apache Flink [2, 14] inject barrier markers into the data stream to capture asynchronous distributed snapshots that establish a consistent recovery boundary accounting for in-flight messages, and replay input logs on failure for exactly-once semantics. Alternatively, Spark uses lineage-based micro-batch checkpointing [3, 57], while Storm and Heron use tuple acknowledgment and replay [32, 54]. Data-intensive middleware can cache compact reduction objects and redistribute unfinished work after failures [11]. These mechanisms decouple operator recovery from overall latency, supporting high throughput under transient faults. However, such approaches are often too conservative in that checkpoint capture is subject to slow serialization and synchronous I/O, while replay is done in a rigid fashion, without load-balancing or straggler mitigation strategies, which ultimately propagates delays through the entire workflow both during failure-free and faulty regimes of operation.

Mitigation of Non-Fail-Stop Anomalies and Gray Failures.

A critical frontier in resilience research involves the management of non-fail-stop anomalies, often referred to as “gray failures”, where nodes remain operational but exhibit sub-optimal performance [29]. These issues stem from resource contention, network jitter, or software misconfigurations, and are particularly disruptive because they propagate delays through downstream dependencies, leading to significant tail latencies [17]. Unlike fail-stop errors, these anomalies cannot be resolved by simple restarts. Instead, modern resilience frameworks employ proactive mitigation techniques, such as speculative execution [1, 18] (launching redundant “straggler” tasks) and dynamic load balancing [24, 36, 50]. By utilizing observability-driven feedback loops, these systems detect performance deviations and rebalance workloads in real-time, preventing a localized slowdown from cascading into a systemic bottleneck. However, such techniques are often oblivious to end-to-end QoS objectives and latency considerations.

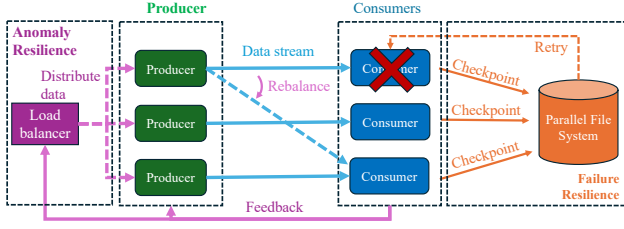


Figure 2: Resilience solutions: (i) Checkpoint and retry for failure resilience and (ii) dynamic load-balancing for anomaly resilience

5 StreamGuard: Resilient HPC Streaming

We propose **StreamGuard**, a resilience solution that combines two complementary mechanisms to protect against failures and performance anomalies while preserving the performance characteristics required by real-time scientific streaming workflows.

5.1 Overall Design

We design resilience mechanisms that decouple resilience from workflow-specific execution frameworks while explicitly targeting real-time scientific streaming requirements. Figure 2 illustrates the overall architecture of StreamGuard. The central design principle is *modularity*. Rather than embedding resilience into a specific execution framework or enforcing workflow-wide coordination, resilience is provided as independent modules that can be selectively applied to different parts of a streaming workflow. This design improves portability and allows scientific workflows to integrate resilience support without modifying their execution model.

Instead of operating at the workflow level, we focus on producer–consumer pairs, which form the fundamental building blocks of scientific streaming applications. Most computation and performance sensitivity occur at these boundaries, where data are produced, processed, and forwarded. Operating at this granularity avoids unnecessary global coordination and allows resilience mechanisms to be applied only where needed, reducing overhead while maintaining flexibility. For each producer–consumer pair, we provide two complementary resilience mechanisms that independently address failures and performance anomalies.

The *failure resilience* mechanism mitigates worker failures through a lightweight checkpoint and retry process. Consumer workers periodically capture snapshots of their processing state and persist them as checkpoints on the parallel file system. Upon failure, the execution module automatically restarts affected workers on available resources and restores the most recent checkpointed state, allowing execution to resume from a consistent point rather than restarting from the beginning. This significantly reduces recovery time and limits disruption to ongoing stream processing.

The *anomaly resilience* mechanism addresses slowdown anomalies caused by performance heterogeneity or resource contention. Using runtime feedback from consumers, the module continuously monitors progress on data partitions and estimates effective worker processing capacity. When imbalance is detected, a load balancer dynamically adjusts partition-to-worker assignments by migrating slower partitions to faster workers. This maintains balanced

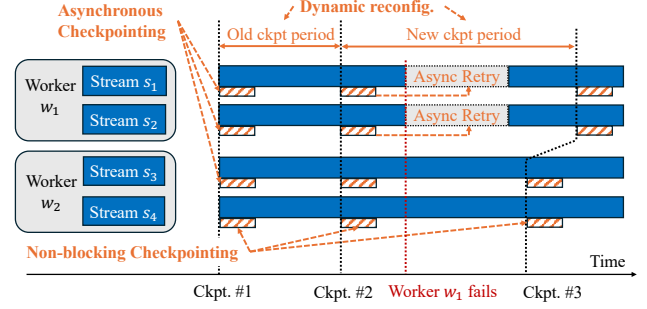


Figure 3: Checkpoint and retry example with two workers and four partitions

progress across partitions and helps ensure that end-to-end processing remains within real-time constraints.

Together, these modules provide resilience against both failures and performance anomalies while avoiding tight coupling between mechanisms or reliance on global coordination. This separation prevents failures and slowdowns from compounding and enables robust execution across a wide range of operating conditions. The detailed design and implementation of each module are described in the following subsections.

5.2 Key Design Principles

5.2.1 Decoupled Lightweight Checkpoint-Restart. We design a lightweight checkpoint-restart mechanism that leverages the embarrassingly parallel structure of scientific streaming workflows. The design is based on three key principles.

Asynchronous, Per-Partition Checkpointing. Instead of maintaining a single global checkpoint, we construct checkpoints independently for each data partition s_i . Each such checkpoint is self-contained and includes all information required to restore processing progress, such as intermediate state and consumed records. Because streams are processed independently, workers can checkpoint without coordinating with other workers. This eliminates global synchronization overhead and simplifies recovery, allowing failed workers to resume execution from their most recent consistent state while unaffected workers continue processing normally.

Non-blocking Execution. Scientific stream processing is typically computation-intensive, whereas checkpoint creation is I/O-intensive. This allows us to remove checkpoint creation from the execution critical path by overlapping the two processes. When a checkpoint is triggered, the worker provides references to the memory regions containing checkpoint data and immediately resumes computation, while a separate process persists the checkpoint in the background. By overlapping computation with I/O, the system improves resource utilization and keeps checkpoint overhead negligible during failure-free execution.

Dynamic Checkpoint Period. A central challenge in checkpoint-based resilience is determining how frequently checkpoints should be created. Frequent checkpointing reduces recomputation after

failures but increases runtime overhead, whereas infrequent checkpointing minimizes overhead but increases recovery cost. Classical solutions such as the Young/Daly formula [16, 56] provide an optimal checkpoint interval under static assumptions. However, scientific streaming environments often experience fluctuating failure rates and variable I/O performance due to slowdown anomalies.

To address this, we dynamically adjust the checkpoint period for each worker based on observed runtime behavior. Specifically, each worker w_j computes its checkpoint period based on a modified Young/Daly formula:

$$W(w_j) = \sqrt{2 \frac{\mu}{M} C(w_j)}, \quad (5)$$

where M is the number of workers, μ the MTTF, and $C(w_j)$ the empirically observed per-worker checkpoint overhead. Each worker computes $W(w_j)$ independently from its own $C(w_j)$ and reconfigures it after failures or when checkpoint overhead changes, allowing the system to adapt to changing I/O and execution conditions while minimizing checkpoint overhead and recomputation cost.

Figure 3 illustrates the mechanism with four partitions across two workers, each creating its own checkpoint asynchronously in the background. When a worker fails, only the partitions assigned to that worker stop execution, while surviving workers continue processing unaffected. The failed partitions are later recovered using their most recent checkpoints, avoiding costly recomputation. Because checkpoints are independent across streams, recovery requires no inter-worker synchronization, allowing failure handling to remain lightweight and scalable.

Algorithm 1 Load-balancing Algorithm

```

1:  $\beta \leftarrow \frac{N}{M}$ 
2: while true do
3:   Load balancing step  $t \leftarrow t + 1$ 
4:   Update  $p_i(t)$  for all partitions  $s_i$ 
5:   Calculate  $c_j$  for all workers  $w_j$  with Equation 7
6:   Compute  $G(t)$  by Equation 6
7:   if  $G(t) < G_{threshold}$  then
8:      $s_{min} \leftarrow \arg \min_{s_i} p_i(t)$  ▷ Slowest partition
9:      $c_{min} \leftarrow s_{min}$ 's worker capacity
10:    for all workers  $w_j$  do
11:       $surplus_j \leftarrow c_j(t) - \sum_i a(s_i, w_j, t)$ 
12:    end for
13:    sort  $w_j$  by decreasing order of  $c_j$ 
14:    partition_moved  $\leftarrow$  false
15:    for each worker  $w_j$  in sorted order do
16:      if  $surplus_j > 1$  and  $c_j > c_{min}$  then
17:        Reassign  $s_{min}$  to  $w_j$ 
18:        partition_moved  $\leftarrow$  true
19:      break
20:    end if
21:  end for
22:  if not partition_moved then
23:     $s_{max} = \arg \max_{s_i} p_i(t)$ 
24:    swap  $s_{min}$  and  $s_{max}$ 
25:  end if
26: end if
27:    $G_{threshold} \leftarrow G$ 
28: end while

```

5.2.2 Load Balancing. To mitigate degradation anomalies, we propose a dynamic load-balancing algorithm that continuously tracks per-partition processing progress and redistributes partitions across workers according to their effective processing speed. The objective is to prevent slow workers from violating the real-time deadline. Consider a producer-consumer pair operating within a real-time deadline window $[0, L]$, where 0 is the time when the first record is emitted by the producer and L is the processing deadline. Let $p_i(t)$ denote the progress of partition s_i at time $t \in [0, L]$. We define

$$p_{\min}(t) = \min_i p_i(t), \quad p_{\max}(t) = \max_i p_i(t)$$

as the slowest and fastest progress on data partitions at time t , respectively. From the real-time constraint (Equation 3), the workflow meets the deadline only if the slowest partition completes on time, i.e., $p_{\min}(L) \geq P$. This implies that overall completion time is determined solely by the slowest partition. Consequently, improving real-time performance requires prioritizing the progress of lagging partitions, even if doing so slightly slows faster ones because the number of available workers is limited to M . To quantify imbalance among partitions, we define the *progress ratio*, $G(t)$, as:

$$G(t) = \frac{p_{\min}(t)}{p_{\max}(t)}, \quad (6)$$

which measures how evenly progress is distributed across partitions. When $G(t) = 1$, all partitions advance at the same rate. The goal of

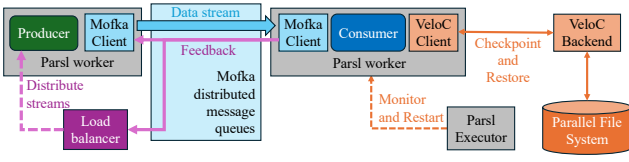


Figure 4: StreamGuard implementation for an individual producer-consumer building block with four resilience modules: (i) resilience execution (gray), (ii) resilience communication (blue), (iii) failure protection (orange), and (iv) slowdown protection (purple)

load balancing is therefore to maintain $G(t)$ close to 1 by reducing progress disparity among partitions.

We achieve this objective by identifying worker performance dynamically using observed execution behavior. Let the *worker active time* $v_j(t)$ denote the total time during which worker w_j is alive and assigned at least one partition. Let $q_j(t)$ denote the total progress made by that worker (see Equation 2). We define the effective *worker speed* as $r_j(t) = \frac{q_j(t)}{v_j(t)}$, which captures the average processing rate of the worker during execution. Partitions associated with slower workers are candidates for reassignment to faster workers.

However, assigning too many partitions to a fast worker may overload it and degrade overall performance. To avoid this, we introduce a capacity model. Recall that N is the total number of partitions and M the number of workers. The ideal number of partitions per worker under homogeneous conditions is $\beta = \frac{N}{M}$. We adjust this quantity for worker speed to obtain the worker's capacity, $c_j(t)$:

$$c_j(t) = \beta \frac{r_j(t)}{\frac{1}{M} \sum_j r_j(t)}, \quad (7)$$

which represents the number of partitions a worker w_j can sustain relative to the average worker. A slow partition is reassigned only to workers whose current load is below their capacity. If no such worker exists, the algorithm swaps the slowest partition with the fastest one, sacrificing some progress of the fastest stream in order to improve the global progress balance.

The complete procedure is summarized in Algorithm 1. The algorithm operates periodically: partition progress is updated at fixed intervals, after which the progress ratio $G(t)$ is evaluated. If $G(t)$ falls below a predefined threshold $G_{\text{threshold}}$, reassignment is triggered (Line #8). The algorithm selects the slowest partition ($s_{\min}$, Line #9) and attempts to migrate it to the fastest worker with available capacity (Lines #10–#21). When no worker has spare capacity, the slowest and fastest partitions are swapped to improve balance. Through repeated adjustments, the algorithm gradually equalizes progress across streams, preventing slowdown anomalies from propagating to end-to-end processing time.

6 StreamGuard Implementation

We implement StreamGuard as a software development kit (SDK) that provides resilience capabilities as independent, composable modules. Applications enable only the mechanisms they need, while

the SDK design avoids tight coupling with specific execution frameworks, allowing integration into existing workflows without modifying their execution model. The overall architecture of StreamGuard is illustrated in Figure 4 with four resilience modules, each responsible for a distinct aspect of reliable stream processing.

Resilience Execution Module. This module provides reliable execution by automatically restarting failed workers on available compute resources. We implement this functionality using Parsl [5] (gray boxes in Figure 4). Each stream processing task is implemented as a function whose inputs are records from a given partition s_i and outputs are data to be forwarded downstream. The function is executed inside a Parsl-managed container (i.e., the worker w_j in our resilience model), allowing Parsl to monitor worker liveness and automatically relaunch failed workers while preserving identifiers and stream mappings. The execution module exposes runtime information, including worker status, failure events, and resource availability, to other resilience modules through a resilient communication layer (described below). This information enables checkpointing and load-balancing decisions to adapt dynamically during execution.

Resilience Communication Module. This module provides reliable producer-consumer communication via a publish/subscribe distributed queue (blue boxes in Figure 4), offering two benefits. First, the queue provides reliable transmission with QoS guarantees (e.g., exactly-once), letting streams tolerate network failures without application-level recovery logic. Second, data persistence in the queue is decoupled from worker execution (workers retrieve data only when ready), preventing failures and slowdowns from propagating across the workflow.

We use Mofka [21, 27], a high-performance distributed message queue designed for HPC environments, as the underlying communication layer. StreamGuard provides a wrapper that allows applications to define logical data streams using unique keys, where each key k_i corresponds to a data partition s_i mapped to a dedicated Mofka topic. Producers publish records to these topics, which are subscribed by consumers for transmission. This abstraction simplifies application logic and enables other resilience modules to observe partition progress and exchange runtime information transparently and reliably.

Failure Protection Module. This module protects workflow state against failures through the lightweight checkpointing mechanism described in Section 5.2.1. The implementation is based on a customized version of VeloC [43], a multi-level checkpoint runtime specialized for HPC infrastructure and large-scale deployment. Checkpointing is organized per producer-consumer pair. Each checkpoint set represents the state of a consumer, where entries correspond to individual data partitions identified by their keys. During initialization, applications register checkpointable state and receive checkpoint handlers that can be invoked during execution.

During normal execution, each worker w_j issues checkpoint requests after reaching consistent processing points, typically at the end of an iteration. If the elapsed time since the previous checkpoint exceeds the current checkpoint period $W(w_j)$, the in-memory

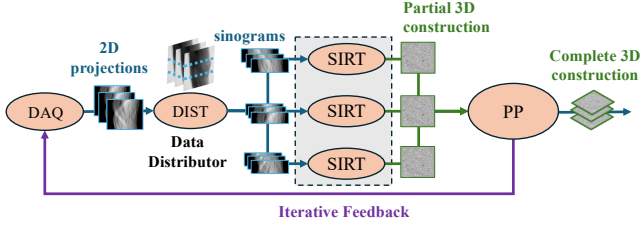


Figure 5: The Tomographic Reconstruction Workflow

state is passed to a background VeloC process and written asynchronously, allowing computation to continue without interruption. Since checkpoints for different partitions are handled independently, the mechanism avoids global synchronization and introduces no inter-worker communication overhead. The module continuously monitors checkpoint latency and observed failure frequency to dynamically adjust the checkpoint period according to Equation 5, maintaining near-optimal overhead.

A worker may also explicitly enforce checkpointing when required by execution semantics, for example, after reaching a critical processing milestone or prior to reassigning a partition to another worker during load balancing. In such cases, the SDK issues a blocking checkpoint request that forces VeloC to complete the checkpoint before control is returned to the worker. This ensures that the worker state is fully persisted before execution proceeds, preventing progress loss during reassignment or recovery.

Slowdown Protection Module. This module mitigates slowdown anomalies using the dynamic load-balancing mechanism described in Section 5.2.2 (purple boxes in Figure 4). Its objective is to maintain balanced progress across streams despite heterogeneous worker performance. The module monitors partition progress through the communication module by tracking per-partition record consumption rates at consumer workers and estimating effective processing speed.

Applications may optionally enable this module for selected producer-consumer pairs. When an imbalance is detected, the load balancer initiates partition reassignment by instructing the current consumer to checkpoint and release the affected partition. The partition is then reassigned to a faster worker, which loads the corresponding checkpoint, subscribes to the partition’s topic, and resumes processing. Because reassignment involves only the previous and new consumer, migration proceeds without global synchronization or interruption of other streams, minimizing impact on overall workflow performance.

7 Evaluation

7.1 Methodology

Objectives. The evaluation assesses whether StreamGuard improves reliability in scientific streaming workflows without compromising performance. Our methodology is designed to isolate the impact of resilience mechanisms from application behavior and system noise, allowing us to evaluate both effectiveness and efficiency under controlled conditions. Specifically, we evaluate the following aspects:

- (1) *Resilience Capability*: whether StreamGuard mitigates failures and performance anomalies sufficiently to allow workflows to complete within real-time constraints;
- (2) *Performance Transparency*: whether StreamGuard introduces minimal overhead during failure-free execution;
- (3) *Robustness*: whether StreamGuard provides consistent protection across different scales, failure rates, and anomaly scenarios.

Together, these objectives evaluate whether resilience can be achieved as a practical system property rather than a trade-off against performance.

Metrics. The above objectives are evaluated using metrics reflecting the operational requirements of scientific streaming: (1) *Processing Time*, measured from the first producer emission to the last consumer output, capturing both the resilience and efficiency of the solution; (2) *Deadline Violation*, whether processing time exceeds the maximum acceptable limit; and (3) *Overhead*, the additional time relative to ideal failure-free execution, quantifying performance transparency.

Workload. We evaluate StreamGuard using a real-world scientific streaming workflow based on the Trace reconstruction engine [10, 12], which reconstructs cross-sectional images from X-ray projections collected at synchrotron radiation facilities. This workload is representative of scientific streaming applications due to its iterative computation, strict timing requirements, and sensitivity to failures and performance imbalance.

As shown in Figure 5, the workflow consists of four stages. The data acquisition component (DAQ) collects projection images of size 2048×2048 pixels from synchrotron radiation facilities. The distributor (DIST) normalizes collected data, converts projections into sinograms, and distributes them to parallel reconstruction workers (SIRT), which perform iterative partial 3D reconstruction. The post-processing stage (PP) aggregates partial results into complete reconstructed images.

Our evaluation focuses on the DIST-SIRT producer-consumer pair because it dominates per-iteration cost, and is where reconstruction state is formed, so failures and slowdowns there most directly affect end-to-end latency. To reflect operational constraints, the pair’s outputs must be produced within a fixed deadline. Under normal execution, reconstruction requires approximately 300–350 iterations over about 1500 records, resulting in an average processing time of roughly 200 s. We therefore set the deadline to 1000 s, providing sufficient recovery margin while preserving realistic real-time requirements.

Baselines. We compare StreamGuard against baselines covering both HPC-style and stream-based resilience mechanisms:

- (1) *Original Implementation*. The unmodified workflow without resilience mechanisms. This configuration represents the performance upper bound under failure-free execution but provides no protection against failures or anomalies.
- (2) *Original + VeloC (X)*. The original workflow augmented with VeloC [43] checkpointing every X iterations; a lightweight, state-of-the-art HPC baseline that enables comparison of static against dynamic checkpoint policies.

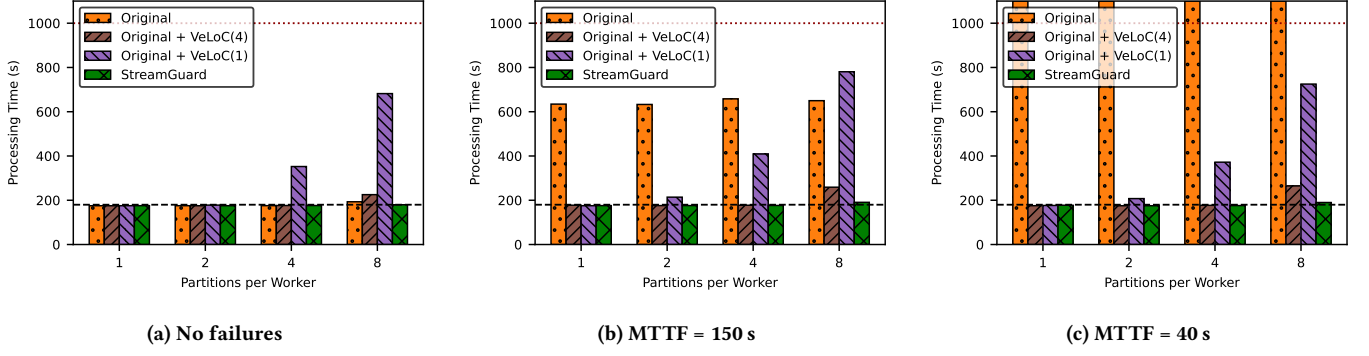


Figure 6: Dynamic checkpointing efficiency. StreamGuard maintains processing time close to ideal execution (black dashed line) and consistently meets the real-time deadline (red dotted line) across varying computation loads and failure scenarios. In contrast, the original implementation fails to meet the deadline under failures, while static checkpointing depends on preconfigured intervals, resulting in inconsistent performance as failure rates and workloads change.

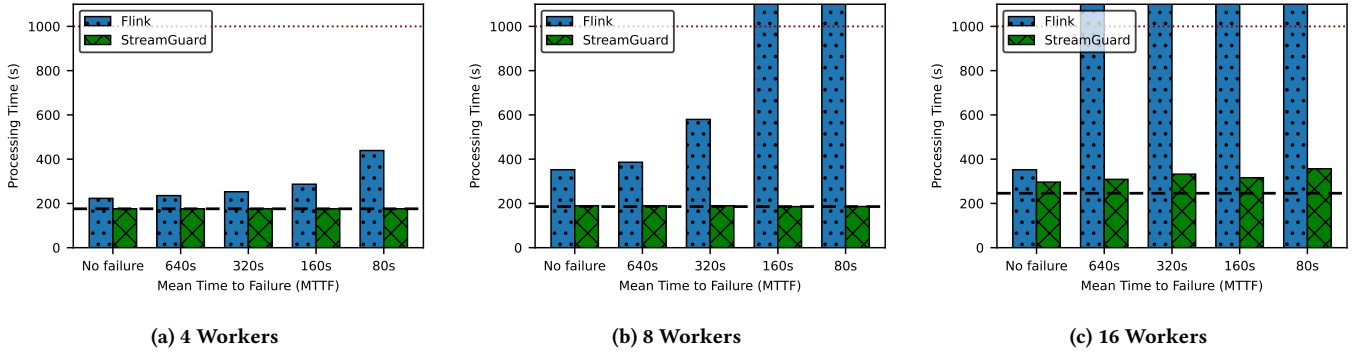


Figure 7: Non-blocking asynchronous checkpoint efficiency. StreamGuard allows each consumer (reconstruction worker) to perform checkpoints independently and overlap checkpointing with computation, resulting in negligible overhead (with respect to ideal processing time, dashed black lines) and enabling real-time deadlines (red dotted lines) to be met across scales, even under very frequent failures. In contrast, Flink requires coordinated checkpointing through barrier propagation across workers, which increases synchronization overhead and causes deadline violations at high failure rates and large scales.

- (3) *Flink*. The workflow is implemented using Apache Flink, a widely used stream processing engine with built-in fault tolerance for stateful operators. Flink maintains resilience by inserting checkpoint barriers into data streams; workers pause processing upon receiving barriers, snapshot their state, and propagate barriers downstream until a globally consistent checkpoint is established. Since Flink does not natively restart failed workers, we implement an external monitoring mechanism to detect failures and relaunch workers when necessary. We further tune checkpointing modes, data partitioning, and parallelism to obtain Flink’s best-performing configuration for fair comparison.
- (4) *StreamGuard*. Implementing our resilient mechanisms as described in Section 6. Although the resilience components are designed as independent modules, they are enabled together during evaluation whenever applicable. This allows us to assess their combined impact in realistic deployment scenarios, capturing both the overall resilience benefits and the cumulative overhead introduced by the complete solution.

System Configurations. All experiments are conducted on the Polaris supercomputer at Argonne National Laboratory. To minimize external interference, the workflow is deployed on dedicated compute nodes without co-located applications. Each node contains a single AMD EPYC “Milan” processor with 64 physical cores and 512 GB of memory, interconnected via a 200 Gbps high-speed network. Checkpoints are stored on the Lustre parallel file system, enabling shared access during recovery.

Failures and anomalies are injected using independent modules that operate transparently to the workflow. For failure injection, a daemon periodically terminates active workers by sending kill signals, with inter-failure intervals drawn from an exponential distribution, which is commonly used to model real-world failure behavior [20]. The distribution is parameterized by a predefined mean time to failure (MTTF), allowing controlled evaluation under different failure intensities.

To emulate performance anomalies, each worker is assigned an artificial slowdown that adds additional computation before

processing each record. The slowdown magnitude varies across workers and is sampled from an exponential distribution, commonly used to approximate the power-law behavior observed in performance variability and straggler effects [17]. The distribution is parameterized by a predefined mean at the start of each run, and the assigned slowdown remains fixed for the worker’s lifetime.

7.2 Experimental Results

Checkpoint and Retry Efficiency. First, we evaluate the effectiveness of StreamGuard’s checkpoint and retry mechanism in protecting streaming data processing against unexpected failures and compare its performance with state-of-the-art solutions from both HPC and industrial stream-processing domains, represented by VeloC and Flink, respectively.

Figure 6 reports consumer processing time as we vary the number of data partitions mapped to each worker under three conditions: failure-free execution, light failures (per-worker MTTF = 150 s), and heavy failures (MTTF = 40 s). The key observation is that StreamGuard’s asynchronous, non-blocking checkpoint mechanism introduces negligible overhead in failure-free runs. As shown in Figure 6a, the overhead remains below 1%, and the completion time is nearly identical to execution without checkpointing (green bars). This demonstrates that resilience can be achieved without sacrificing performance when failures do not occur.

In contrast, the checkpoint-free configuration in the original implementation (orange bars) is highly vulnerable to failures. Each failure forces a worker to restart from the beginning, discarding previously completed computation and significantly increasing total processing time. Even under a light failure rate (MTTF = 150 s), recomputation overhead becomes substantial (Figure 6b). Under heavy failures (MTTF = 40 s, Figure 6c), the accumulated recomputation cost becomes prohibitive, preventing the workflow from completing within the deadline window (1000 s).

Static checkpointing cannot fully resolve this problem. Frequent checkpointing (Original+VeloC(1), purple bars) sustains progress under heavy failures but incurs excessive overhead when failures are infrequent, exceeding 3× at eight partitions per worker due to I/O contention from concurrent checkpoints. Reducing frequency (Original+VeloC(4), brown bars) lowers failure-free overhead but becomes ineffective at high failure rates. A fixed interval cannot adapt to changing failure rates and load. In contrast, StreamGuard adapts checkpoint frequency to observed cost and failure behavior, maintaining low overhead and resilience across all scenarios.

Figure 7 compares StreamGuard with Flink, which employs synchronized, barrier-based checkpointing. This mechanism places checkpoint cost directly on the processing critical path because workers must pause while snapshots are taken. Moreover, checkpoint progress depends on barrier propagation across all workers; under failures or performance imbalance, slower workers delay barrier completion, causing congestion and backpressure. As shown in Figure 7a, Flink execution is already 25% slower than the original implementation under failure-free conditions, and processing time increases further as failure frequency grows. The degradation is magnified at scale: at 16 workers with per-worker MTTF of 640 s, the SIRT stage experiences disruptions approximately every

40 s, preventing checkpoint barriers from reliably traversing the workflow and causing deadline violations.

In contrast, StreamGuard performs checkpoints in a fully non-blocking and asynchronous manner, removing checkpoint overhead from the processing critical path. In addition, data are streamed through a dedicated queueing service (Mofka), allowing workers to fetch data on demand and decoupling resilience behavior from flow control and data movement. As a result, checkpoint progress remains unaffected by transient slowdowns or failures in other workers. Across all evaluated scales and failure settings in Figure 7, StreamGuard consistently meets real-time deadlines. Even under the most extreme case with 16 workers and MTTF = 80 s, which is equivalent to one failure every five seconds within the SIRT stage, the processing time remains within 20% of the ideal execution.

Takeaway: *Dynamic, non-blocking, asynchronous checkpoint and retry minimizes failure impact while preserving near-ideal performance, enabling robust real-time processing, even under extreme conditions, where static or synchronized checkpointing fails.*

Dynamic Load Balancing. Next, we evaluate the effectiveness of StreamGuard’s dynamic load-balancing algorithm in mitigating the impact of slowdown anomalies. To emulate heterogeneous worker performance, we deploy the workflow with injected slowdown using the anomaly model described in Section 7.1. Figure 8 reports the processing time of the workflow under StreamGuard compared with Flink and the original implementation across three slowdown configurations: *light* (average slowdown of 500 ms), *medium* (1000 ms), and *heavy* (2000 ms), and across three deployment scales of 4, 8, and 16 workers.

Both Flink and the original implementation rely on static partition-to-worker mappings. As a result, end-to-end processing time is dictated by the slowest worker. When slowdown anomalies are present, this leads to substantial performance degradation, with processing time increasing by at least 3×. The impact becomes more pronounced as the number of workers increases, since a larger deployment increases the likelihood of encountering slower workers that delay overall progress. Under medium and heavy slowdown conditions (1000 ms and 2000 ms), the accumulated delay prevents both approaches from completing processing before the deadline, even at small scale with only four workers.

In contrast, StreamGuard continuously monitors partition progress and estimates effective worker speed during execution. Lagging partitions are proactively migrated to faster workers, allowing the system to rebalance workload dynamically as performance imbalance emerges. This dynamic rebalancing progressively equalizes progress across partitions and prevents slow workers from dominating overall execution time. As a result, with four workers, processing time increases by at most 3× while still meeting the real-time deadline, whereas both Flink and the original implementation fail to complete on time. Furthermore, as the number of workers increases, processing time remains stable because additional faster workers can compensate for slower ones. This allows StreamGuard to maintain nearly constant overhead across scales and consistently satisfy real-time constraints.

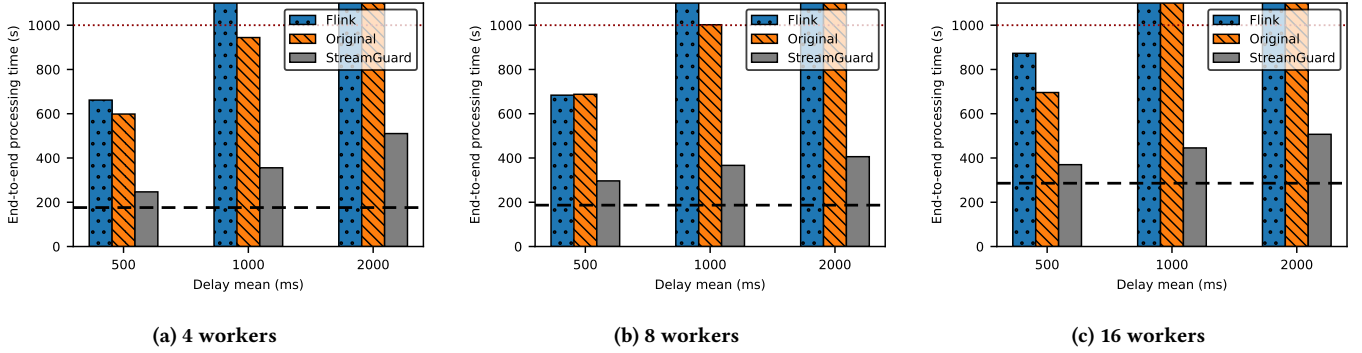


Figure 8: Dynamic load-balancing efficiency. Load rebalancing based on worker speed and per-partition progress significantly reduces processing lag caused by slow workers, enabling real-time constraints to be satisfied across scales and anomaly scenarios where existing solutions fail. The black dotted line indicates the processing time of the original implementation under failure-free (ideal) execution, while the red dotted line denotes the processing deadline.

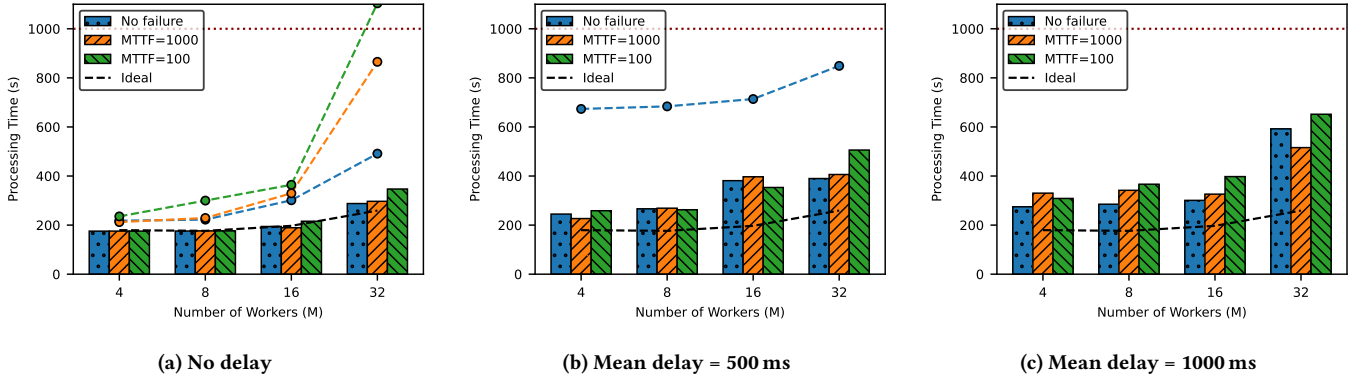


Figure 9: Resilience robustness. StreamGuard maintains acceptable processing time and satisfies all real-time deadlines across failure rates, anomaly conditions, and scales. Bars show StreamGuard processing time; lines show Flink processing time. Colors indicate failure settings. Missing lines indicate Flink runs that exceed the deadline.

Takeaway: *Dynamic load balancing effectively mitigates slowdown anomalies by carefully reassigning slow partitions to fast workers, enabling robust real-time processing across scales and heterogeneous execution conditions.*

Robustness. Finally, we evaluate the robustness of StreamGuard by executing the workflow under combined failure and slowdown anomaly conditions at multiple scales, and comparing the results against both Flink and the ideal execution. The results are summarized in Figure 9. We consider three anomaly settings: no slowdown (Figure 9a), an average slowdown of 500 ms (Figure 9b), and an average slowdown of 1000 ms (Figure 9c). Each anomaly setting is evaluated under three failure configurations: no failures (blue), per-worker MTTF of 1000 s (orange), and MTTF of 100 s (green), while scaling the deployment from four to 32 workers. Bars represent the processing time of StreamGuard, while lines represent Flink processing time.

Consistent with observations from previous experiments, Flink’s synchronized checkpointing mechanism makes it increasingly sensitive to high failure rates, particularly at larger scales where the

probability of failure increases with the number of workers. As scale and failure frequency increase, checkpoint coordination overhead grows, resulting in longer processing times. The presence of slowdown anomalies further amplifies this effect by extending processing duration, which in turn exposes Flink to additional failures within the same execution window. This compounding effect leads to rapidly increasing overhead, causing Flink to miss the processing deadline in most configurations once the average slowdown reaches 500 ms.

In contrast, StreamGuard maintains stable performance across all tested configurations. Failures and slowdown anomalies are handled by separate resilience mechanisms with minimal coupling, preventing their negative effects from amplifying each other. Dynamic checkpointing limits recovery overhead under frequent failures, while load balancing mitigates processing lag caused by slow workers. As a result, processing time remains close to the ideal execution even as scale, failure rate, and anomaly severity increase, allowing the workflow to consistently meet real-time deadlines across all evaluated settings.

Takeaway: By handling failures and anomalies independently with minimal cross-dependency, StreamGuard prevents their effects from compounding, enabling robust real-time execution across a range of failure rates, anomaly conditions, and deployment scales.

8 Conclusions and Future Work

We presented StreamGuard, a resilience architecture for real-time scientific streaming workflows. By treating the producer-consumer pair as the unit of resilience and integrating dynamic non-blocking checkpointing with progress-aware load balancing as decoupled, composable modules, StreamGuard avoids global coordination while maintaining real-time deadlines under frequent failures, persistent slowdown anomalies, and increasing scale. Experiments on a real-world tomographic reconstruction workflow show that resilience can be achieved as a first-class system property without sacrificing scalability or performance transparency.

Several promising directions extend this work. First, StreamGuard assumes approximately uniform per-partition computational cost, which may not always hold in practice. Extending the load balancer's capacity model with per-partition cost weights would broaden applicability to skewed workloads. Second, StreamGuard is evaluated with sufficient allocated resources for the experimental workload. Yet in practice, load spikes or miscalculations can cause transient under- or over-provisioning. Although our prior work has studied resource-workload mismatch [40], its setting does not target real-time streaming; studying such scenarios in a streaming context would help develop more robust and practical solutions. Finally, while the mechanisms presented here are stage-local by design, evaluating compound effects across multiple producer-consumer stages, including cross-stage I/O interference and backpressure, remains an important direction for future work.

Acknowledgments

This work was supported in part by the U.S. Department of Energy under Contract DE-AC02-06CH11357, as well as the National Science Foundation (NSF), Office of Advanced Cyberinfrastructure, under grants CSSI-2411386 and CSSI-2514056.

References

- [1] Ganesh Ananthanarayanan, Srikanth Kandula, Albert Greenberg, Ion Stoica, Yi Lu, Bikas Saha, and Edward Harris. 2010. Reining in the outliers in {Map-Reduce} clusters using mantri. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*.
- [2] Apache Flink. [n.d.]. Checkpoints. <https://nightlies.apache.org/flink/flink-docs-stable/docs/ops/state/checkpoints/>. Apache Flink Documentation, accessed: 2026-02-09.
- [3] Michael Armbrust, Tathagata Das, Joseph Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, and Matei Zaharia. 2018. Structured streaming: A declarative api for real-time applications in apache spark. In *Proceedings of the 2018 International Conference on Management of Data*. 601–613.
- [4] Anakha V Babu, Tao Zhou, Saugat Kandel, Tekin Bicer, Zhengchun Liu, William Judge, Daniel J Ching, Yi Jiang, Sinisa Veseli, Steven Henke, et al. 2023. Deep learning at the edge enables real-time streaming ptychographic imaging. *Nature Communications* 14, 1 (2023), 7059.
- [5] Yadu Babuji, Anna Woodard, Zhuozhao Li, Daniel S Katz, Ben Clifford, Rohan Kumar, Lukasz Laciniski, Ryan Chard, Justin M Wozniak, Ian Foster, et al. 2019. Parsl: Pervasive parallel programming in python. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*. 25–36.
- [6] GAUDI & Barrand, I Belyaev, P Binko, M Cattaneo, R Chytracsek, G Corti, M Frank, G Gracia, J Harvey, Eric Van Herwijnen, et al. 2001. GAUDI—A software architecture and framework for building HEP data processing applications. *Computer physics communications* 140, 1-2 (2001), 45–55.
- [7] Leonardo Bautista-Gomez, Seiji Tsuboi, Dimitri Komatitsch, Franck Cappello, Naoya Maruyama, and Satoshi Matsuoka. 2011. FTI: High Performance Fault Tolerance Interface for Hybrid Systems. In *ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*. Seattle, USA, 32:1–32:32.
- [8] Anne Benoit, Yishu Du, Thomas Herault, Loris Marchal, Guillaume Pallez, Lucas Perotin, Yves Robert, Hongyang Sun, and Frédéric Vivien. 2022. Checkpointing à la Young/Daly: An overview. In *14th International Conference on Contemporary Computing*. 701–710.
- [9] Anne Benoit, Lucas Perotin, Yves Robert, and Frédéric Vivien. 2024. Checkpointing Strategies to Tolerate Non-Memoryless Failures on HPC Platforms. *ACM Trans. Parallel Comput.* 11, 1, Article 1 (2024), 26 pages.
- [10] Tekin Bicer, Doga Gürsoy, Vincent De Andrade, Rajkumar Kettimuthu, William Scullin, Francesco De Carlo, and Ian T Foster. 2017. Trace: A high-throughput tomographic reconstruction engine for large-scale datasets. *Advanced Structural and Chemical Imaging* 3 (2017), 1–10.
- [11] Tekin Bicer, Wei Jiang, and Gagan Agrawal. 2010. Supporting fault tolerance in a data-intensive computing middleware. In *2010 IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*. IEEE, 1–12.
- [12] Tekin Bicer, Viktor Nikitin, Selin Aslan, Doga Gürsoy, Rajkumar Kettimuthu, and Ian T Foster. 2020. Tomographic Reconstruction of Dynamic Features with Streaming Sliding Subsets. In *Annual Workshop on Large-scale Experiment-in-the-Loop Computing*. IEEE/ACM.
- [13] Franck Cappello, Al Geist, William Gropp, Sanjay Kale, Bill Kramer, and Marc Snir. 2014. Toward exascale resilience: 2014 update. *Supercomputing Frontiers and Innovations: An International Journal* 1, 1 (2014), 5–28.
- [14] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering* 38, 4 (2015).
- [15] K Mani Chandy and Leslie Lamport. 1985. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems (TOCS)* 3, 1 (1985), 63–75.
- [16] John T Daly. 2006. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems* 22, 3 (2006), 303–312.
- [17] Jeffrey Dean and Luiz André Barroso. 2013. The tail at scale. *Commun. ACM* 56, 2 (2013), 74–80.
- [18] Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: simplified data processing on large clusters. *Commun. ACM* 51, 1 (2008), 107–113.
- [19] Martin Dierolf, Andreas Menzel, Pierre Thibault, Philipp Schneider, Cameron M Kewish, Roger Wepf, Oliver Bunk, and Franz Pfeiffer. 2010. Ptychographic X-ray computed tomography at the nanoscale. *Nature* 467, 7314 (2010), 436–439.
- [20] Jack Dongarra, Thomas Herault, and Yves Robert. 2015. *Fault tolerance techniques for high-performance computing*. Springer.
- [21] Matthieu Dorier, Amal Gueroudji, Valérie Hayot-Sasson, Hai Duc Nguyen, Seth Ockerman, Renan Souza, Tekin Bicer, Haochen Pan, Philip Carns, Kyle Chard, et al. 2025. Toward a persistent event-streaming system for high-performance computing applications. *Frontiers in High Performance Computing* 3 (2025), 1638203.
- [22] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [23] Kurt Ferreira, Jon Stearley, James H Laros III, Ron Oldfield, Kevin Pedretti, Ron Brightwell, Rolf Riesen, Patrick G Bridges, and Dorian Arnold. 2011. Evaluating the viability of process replication reliability for exascale systems. In *International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–12.
- [24] Michael Goodrich, Carl Timmer, Vardan Gyurjyan, David Lawrence, Graham Heyes, Yatish Kumar, and Stacey Sheldon. 2023. ESnet/JLab FPGA Accelerated Transport. *IEEE Transactions on Nuclear Science* 70, 6 (2023), 1096–1101.
- [25] Google Cloud. 2026. Google Cloud Dataflow. <https://cloud.google.com/products/dataflow>. Accessed: 2026-02-09.
- [26] Mikaila J. Gossman, Bogdan Nicolae, and Jon C. Calhoun. 2024. Scalable I/O aggregation for asynchronous multi-level checkpointing. *Future Generation Computer Systems* 160 (2024), 420–432. doi:10.1016/j.future.2024.06.003
- [27] Amal Gueroudji, Matthieu Dorier, Philip Carns, Parth Patel, Tekin Bicer, Robert Latham, Robert Ross, Kyle Chard, and Ian Foster. 2025. RESILIO: A Scalable and Composable Architecture for Tomographic Reconstruction Workflows. In *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*. Association for Computing Machinery, New York, NY, USA, 2281–2292. doi:10.1145/3731599.3767575
- [28] Salman Habib, Vitali Morozov, Nicholas Frontiere, Hal Finkel, Adrian Pope, and Katrin Heitmann. 2013. HACC: Extreme scaling and performance across diverse architectures. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 1–10.
- [29] Peng Huang, Chuanxiong Guo, Jacob R Lorch, Lidong Zhou, and Yingneng Dang. 2017. Gray failure: The Achilles' heel of cloud-scale systems. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*. 150–155.

- [30] Eliu Antonio Huerta, Gabrielle Allen, Igor Andreoni, Javier M Antelis, Etienne Bachelet, G Bruce Berriman, Federica B Bianco, Rahul Biswas, Matias Carrasco Kind, Kyle Chard, et al. 2019. Enabling real-time multi-messenger astrophysics discoveries with deep learning. *Nature Reviews Physics* 1, 10 (2019), 600–608.
- [31] Yongho Kim, Seongha Park, Sean Shahkarami, Rajesh Sankaran, Nicola Ferrier, and Pete Beckman. 2022. Goal-driven scheduling model in edge computing for smart city applications. *J. Parallel and Distrib. Comput.* 167 (2022), 97–108.
- [32] Sanjeev Kulkarni, Nikunj Bhagat, Maosong Fu, Vikas Kedigehalli, Christopher Kellogg, Sailesh Mittal, Jignesh M Patel, Karthik Ramasamy, and Siddarth Taneja. 2015. Twitter heron: Stream processing at scale. In *Proceedings of the 2015 ACM SIGMOD international conference on Management of data*. 239–250.
- [33] Yifei Li, Ryan Chard, Yadu Babuji, Kyle Chard, Ian Foster, and Zhuozhao Li. 2024. UniFaaS: Programming across Distributed Cyberinfrastructure with Federated Function Serving. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 217–229.
- [34] Gang Liu, Zeting Wang, Amelie Chi Zhou, and Rui Mao. 2024. Adaptive key partitioning in distributed stream processing. *CCF Transactions on High Performance Computing* 6, 2 (2024), 164–178.
- [35] Zhengchun Liu, Tekin Bicer, Rajkumar Kettimuthu, Doga Gursay, Francesco De Carlo, and Ian Foster. 2020. TomoGAN: Low-dose synchrotron x-ray tomography with generative adversarial networks. *JOSA A* 37, 3 (2020), 422–434.
- [36] Pengqi Lu, Yue Yue, Liang Yuan, and Yunquan Zhang. 2021. AutoFlow: hotspot-aware, dynamic load balancing for distributed stream processing. In *International Conference on Algorithms and Architectures for Parallel Processing*. Springer, 133–151.
- [37] Nicolau Manubens, Johann Lombardi, Simon D. Smart, Emanuele Danovaro, Tiago Quintino, Dean Hildebrand, and Adrian Jackson. 2024. Exploring DAOS Interfaces and Performance. arXiv:2409.18682 [cs.DC] <https://arxiv.org/abs/2409.18682>
- [38] Avinash Maurya, M. Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. 2023. Towards Efficient I/O Pipelines using Accumulated Compression. In *HIPC'23: 30th IEEE International Conference on High Performance Computing, Data, and Analytics*. Goa, India.
- [39] K. Mohror, A. Moody, G. Bronevetsky, and B. R. de Supinski. 2014. Detailed Modeling and Evaluation of a Scalable Multilevel Checkpointing System. *IEEE Transactions on Parallel and Distributed Systems* 25, 09 (sep 2014), 2255–2263. doi:10.1109/TPDS.2013.100
- [40] Hai Duc Nguyen, Tekin Bicer, Bogdan Nicolae, Rajkumar Kettimuthu, Eliu A Huerta, and Ian T Foster. 2025. Resilient execution of distributed X-ray image analysis workflows. *Frontiers in High Performance Computing* 3 (2025), 1550855.
- [41] Hai Duc Nguyen, Zhifei Yang, and Andrew A Chien. 2020. Motivating high performance serverless workloads. In *Proceedings of the 1st Workshop on High Performance Serverless Computing*. 25–32.
- [42] Bogdan Nicolae, Adam Moody, Elsa Gonsiorowski, Kathryn Mohror, and Franck Cappello. 2019. VeloC: Towards High Performance Adaptive Asynchronous Checkpointing at Large Scale. In *IPDPS'19: The 2019 IEEE International Parallel and Distributed Processing Symposium*. Rio de Janeiro, Brazil, 911–920. doi:10.1109/IPDPS.2019.00099
- [43] Bogdan Nicolae, Adam Moody, Greg Kosinovsky, Kathryn Mohror, and Franck Cappello. 2021. VELOC: VERY Low Overhead Checkpointing in the Age of Exascale. In *SuperCheck'21: The First International Symposium on Checkpointing for Supercomputing*. Virtual Event. <https://arxiv.org/pdf/2103.02131.pdf>
- [44] Bogdan Nicolae, Justin M Wozniak, Tekin Bicer, Hai Nguyen, Parth Patel, Haochen Pan, Amal Gueroudji, Maxime Gonther, Valerie Hayot-Sasson, Eliu Huerta, et al. 2024. Diaspora: Resilience-enabling services for real-time distributed workflows. In *2024 IEEE 20th International Conference on e-Science (e-Science)*. IEEE, 1–9.
- [45] Shadi A Noghbi, Kartik Paramasivam, Yi Pan, Navina Ramesh, Jon Bringham, Indranil Gupta, and Roy H Campbell. 2017. Samza: stateful scalable stream processing at LinkedIn. *Proceedings of the VLDB Endowment* 10, 12 (2017), 1634–1645.
- [46] Norbert Podhorszki et al. 2020. ADIOS 2: The Adaptable Input Output System. A framework for high-performance data management. *SoftwareX* 12 (2020). doi:10.1016/j.softx.2020.100529
- [47] Michael Prince, Doga Gursay, Dina Sheyfer, Ryan Chard, Benoit Côté, Hannah Parraga, Barbara Frosik, Jon Tischler, and Nicholas Schwarz. 2023. Demonstrating Cross-Facility Data Processing at Scale With Laue Microdiffraction. In *Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (Denver, CO, USA) (SC-W '23)*. Association for Computing Machinery, New York, NY, USA, 2133–2139. doi:10.1145/3624062.3624613
- [48] Charles Reiss, Alexey Tumanov, Gregory R Ganger, Randy H Katz, and Michael A Kozuch. 2012. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the third ACM symposium on cloud computing*. 1–13.
- [49] Richard D Schlichting and Fred B Schneider. 1983. Fail-stop processors: An approach to designing fault-tolerant computing systems. *ACM Transactions on Computer Systems* 1, 3 (1983), 222–238.
- [50] Won Wook Song, Taegeon Um, Sameh Elnikety, Myeongjae Jeon, and Byung-Gon Chun. 2023. Sponge: Fast reactive scaling for stream processing with serverless frameworks. In *2023 USENIX Annual Technical Conference (USENIX ATC '23)*. 301–314.
- [51] Sebastian Steiner, Jakob Wolf, Stefan Glatzel, Anna Andreou, Jaroslaw M Granda, Graham Keenan, Trevor Hinkley, Gerardo Aragon-Camarasa, Philip J Kitson, Davide Angelone, et al. 2019. Organic synthesis in a modular robotic system driven by a chemical programming language. *Science* 363, 6423 (2019), eaav2211.
- [52] Dawei Sun, Chunlin Zhang, Shang Gao, and Rajkumar Buyya. 2024. An adaptive load balancing strategy for stateful join operator in skewed data stream environments. *Future generation computer systems* 152 (2024), 138–151.
- [53] P Thibault, M Dierolf, A Menzel, O Bunk, C David, and F Pfeiffer. 2008. High-resolution scanning x-ray diffraction microscopy. *Science* 321, 5887 (2008), 379–382.
- [54] Ankit Toshniwal, Siddarth Taneja, Amit Shukla, Karthik Ramasamy, Jignesh M Patel, Sanjeev Kulkarni, Jason Jackson, Krishna Gade, Maosong Fu, Jake Donham, et al. 2014. Storm@ twitter. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 147–156.
- [55] Xiaoli Yan, Nathaniel Hudson, Hyun Park, Daniel Grzenda, J Gregory Pauloski, Marcus Schwarting, Haochen Pan, Hassan Harb, Samuel Foreman, Chris Knight, et al. 2025. MOFA: Discovering Materials for Carbon Capture with a GenAI-and Simulation-Based Workflow. *arXiv preprint arXiv:2501.10651* (2025).
- [56] John W Young. 1974. A first order approximation to the optimum checkpoint interval. *Commun. ACM* 17, 9 (1974), 530–531.
- [57] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. 2013. Discretized streams: Fault-tolerant streaming computation at scale. In *Proceedings of the twenty-fourth ACM symposium on operating systems principles*. 423–438.